```
set -e
P=miya-app
mkdir -p $P/app/src/main/java/com/miya/app $P/app/src/main/assets
cat > $P/settings.gradle <<'MIYA_EOF_0'
pluginManagement {
    repositories { google(); mavenCentral(); gradlePluginPortal() }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories { google(); mavenCentral() }
}
rootProject.name = 'Miya'
include ':app'
MIYA_EOF_0
cat > $P/build.gradle <<'MIYA_EOF_1'
plugins {
    id 'com.android.application' version '8.5.2' apply false
}
MIYA_EOF_1
cat > $P/gradle.properties <<'MIYA_EOF_2'
android.useAndroidX=true
org.gradle.jvmargs=-Xmx2g
MIYA_EOF_2
cat > $P/app/build.gradle <<'MIYA_EOF_3'
plugins {
    id 'com.android.application'
}
android {
    namespace 'com.miya.app'
    compileSdk 34
    defaultConfig {
        applicationId 'com.miya.app'
        minSdk 26
        targetSdk 33
        versionCode 1
        versionName '1.0'
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
}
tasks.withType(JavaCompile) { options.encoding = 'UTF-8' }
dependencies {
    implementation 'androidx.webkit:webkit:1.11.0'
}
```

```
MIYA_EOF_3
cat > $P/app/src/main/AndroidManifest.xml <<'MIYA_EOF_4'
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <queries>
        <intent><action android:name="android.speech.RecognitionService"/></intent>
        <intent><action android:name="android.intent.action.TTS_SERVICE"/></intent>
    </queries>
    <application
        android:label="मिया"
        android:allowBackup="false"
        android:theme="@android:style/Theme.DeviceDefault.Light">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <service android:name=".MiyaService" android:exported="false"/>
    </application>
</manifest>
MIYA_EOF_4
cat > $P/app/src/main/java/com/miya/app/MainActivity.java <<'MIYA_EOF_5'
package com.miya.app;

import android.Manifest;
import android.app.Activity;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.provider.Settings;
import android.view.Gravity;
import android.widget.Button;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        LinearLayout l = new LinearLayout(this);
        l.setOrientation(LinearLayout.VERTICAL);
```

```
        l.setGravity(Gravity.CENTER);
        l.setPadding(60, 60, 60, 60);
        TextView info = new TextView(this);
        info.setTextSize(18);
        info.setText("मिया को आपकी स्क्रीन पर तैरने के लिए ये अनुमतियाँ चाहिए: दूसरे ऐप्स के ऊपर दिखना,
माइक और सूचना। नीचे का बटन दबाइए और हर बार अनुमति दीजिए।");
        Button btn = new Button(this);
        btn.setText("मिया को शुरू करो");
        btn.setOnClickListener(v -> proceed());
        l.addView(info);
        l.addView(btn);
        setContentView(l);
    }

    void proceed() {
        if (!Settings.canDrawOverlays(this)) {
            startActivity(new Intent(Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
Uri.parse("package:" + getPackageName())));
            Toast.makeText(this, "सूची में 'मिया' चुनकर अनुमति दें, फिर वापस आकर बटन दबाएँ",
Toast.LENGTH_LONG).show();
            return;
        }
        if (checkSelfPermission(Manifest.permission.RECORD_AUDIO) !=
PackageManager.PERMISSION_GRANTED) {
            if (Build.VERSION.SDK_INT >= 33) {
                requestPermissions(new String[]{Manifest.permission.RECORD_AUDIO,
"android.permission.POST_NOTIFICATIONS"}, 1);
            } else {
                requestPermissions(new String[]{Manifest.permission.RECORD_AUDIO},
1);
            }
            return;
        }
        startForegroundService(new Intent(this, MiyaService.class));
        Toast.makeText(this, "मिया आ गई! उसे दबाकर बात कीजिए।",
Toast.LENGTH_LONG).show();
        finish();
    }

    @Override
    public void onRequestPermissionsResult(int code, String[] perms, int[] res) {
        super.onRequestPermissionsResult(code, perms, res);
        if (checkSelfPermission(Manifest.permission.RECORD_AUDIO) ==
PackageManager.PERMISSION_GRANTED) {
            proceed();
        } else {
            Toast.makeText(this, "बिना माइक की अनुमति के मिया आपकी बात नहीं सुन पाएगी",
Toast.LENGTH_LONG).show();
        }
    }
```

```
}
MIYA_EOF_5
cat > $P/app/src/main/java/com/miya/app/MiyaService.java <<'MIYA_EOF_6'
package com.miya.app;

import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.Service;
import android.content.Intent;
import android.graphics.Color;
import android.graphics.PixelFormat;
import android.graphics.drawable.Icon;
import android.os.Bundle;
import android.os.Handler;
import android.os.IBinder;
import android.os.Looper;
import android.speech.RecognitionListener;
import android.speech.RecognizerIntent;
import android.speech.SpeechRecognizer;
import android.speech.tts.TextToSpeech;
import android.speech.tts.UtteranceProgressListener;
import android.view.Gravity;
import android.view.MotionEvent;
import android.view.WindowManager;
import android.webkit.WebResourceRequest;
import android.webkit.WebResourceResponse;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;

import androidx.webkit.WebViewAssetLoader;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MiyaService extends Service {
```

```
    static final String WORKER = "https://young-sky-01cdmy-assistant-
api.vbhaimte.workers.dev";
    static final String PERSONA = "तुम \"मिया\" हो, मेरी निजी असिस्टेंट और दोस्त। मुझसे हिंदी में बात
करो (मैं अंग्रेज़ी में बोलूँ तो अंग्रेज़ी में)। "
            + "जवाब छोटे रखो, 1-3 वाक्य, क्योंकि तुम बोलकर जवाब देती हो। इमोजी और markdown मत
इस्तेमाल करो। "
            + "अपनी सीधी और ईमानदार राय दो; मेरी हर बात से सहमत मत होना, ग़लत लगे तो सलीक़े से टोको।";

    WindowManager wm;
    WebView web;
    WindowManager.LayoutParams lp;
    final Handler ui = new Handler(Looper.getMainLooper());
    SpeechRecognizer rec;
    TextToSpeech tts;
    boolean ttsReady = false, speaking = false, busy = false;
    final List<JSONObject> history = new ArrayList<>();
    final ExecutorService net = Executors.newSingleThreadExecutor();
    float downX, downY;
    int startX, startY;
    boolean moved;

    @Override
    public IBinder onBind(Intent i) {
        return null;
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (intent != null && "STOP".equals(intent.getAction())) {
            stopSelf();
            return START_NOT_STICKY;
        }
        if (web == null) {
            startFg();
            buildOverlay();
            initTts();
        }
        return START_STICKY;
    }

    void startFg() {
        NotificationManager nm = getSystemService(NotificationManager.class);
        nm.createNotificationChannel(new NotificationChannel("miya", "मिया",
NotificationManager.IMPORTANCE_LOW));
        Intent stop = new Intent(this, MiyaService.class).setAction("STOP");
        PendingIntent pi = PendingIntent.getService(this, 1, stop,
PendingIntent.FLAG_IMMUTABLE | PendingIntent.FLAG_UPDATE_CURRENT);
        Notification n = new Notification.Builder(this, "miya")
                .setSmallIcon(android.R.drawable.ic_btn_speak_now)
                .setContentTitle("मिया चल रही है")
```

```
                .setContentText("बंद करने के लिए नीचे का बटन दबाएँ")
                .addAction(new Notification.Action.Builder(
                        Icon.createWithResource(this,
android.R.drawable.ic_menu_close_clear_cancel), "बंद करें", pi).build())
                .build();
        startForeground(1, n);
    }

    void buildOverlay() {
        final float d = getResources().getDisplayMetrics().density;
        web = new WebView(this);
        web.setBackgroundColor(Color.TRANSPARENT);
        WebSettings s = web.getSettings();
        s.setJavaScriptEnabled(true);
        s.setDomStorageEnabled(true);
        s.setMediaPlaybackRequiresUserGesture(false);
        final WebViewAssetLoader loader = new WebViewAssetLoader.Builder()
                .addPathHandler("/assets/", new
WebViewAssetLoader.AssetsPathHandler(this))
                .build();
        web.setWebViewClient(new WebViewClient() {
            @Override
            public WebResourceResponse shouldInterceptRequest(WebView v,
WebResourceRequest r) {
                return loader.shouldInterceptRequest(r.getUrl());
            }
        });
        web.loadUrl("https://appassets.androidplatform.net/assets/avatar.html");

        lp = new WindowManager.LayoutParams(
                (int) (230 * d), (int) (320 * d),
                WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY,
                WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE
                        | WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL
                        | WindowManager.LayoutParams.FLAG_HARDWARE_ACCELERATED,
                PixelFormat.TRANSLUCENT);
        lp.gravity = Gravity.TOP | Gravity.START;
        lp.x = 40;
        lp.y = 400;

        web.setOnTouchListener((v, e) -> {
            switch (e.getActionMasked()) {
                case MotionEvent.ACTION_DOWN:
                    startX = lp.x;
                    startY = lp.y;
                    downX = e.getRawX();
                    downY = e.getRawY();
                    moved = false;
                    break;
                case MotionEvent.ACTION_MOVE:
```

```
                    float dx = e.getRawX() - downX, dy = e.getRawY() - downY;
                    if (Math.abs(dx) > 10 * d || Math.abs(dy) > 10 * d) moved = true;
                    if (moved) {
                        lp.x = startX + (int) dx;
                        lp.y = startY + (int) dy;
                        wm.updateViewLayout(web, lp);
                    }
                    break;
                case MotionEvent.ACTION_UP:
                    if (!moved) onTap();
                    break;
            }
            return true;
        });
        wm = (WindowManager) getSystemService(WINDOW_SERVICE);
        wm.addView(web, lp);
    }

    void initTts() {
        tts = new TextToSpeech(this, st -> {
            if (st == TextToSpeech.SUCCESS) {
                tts.setLanguage(new Locale("hi", "IN"));
                tts.setOnUtteranceProgressListener(new UtteranceProgressListener() {
                    @Override
                    public void onStart(String id) {
                        ui.post(() -> { speaking = true; js("setSpeaking(true)"); });
                    }

                    @Override
                    public void onDone(String id) {
                        ui.post(() -> { speaking = false; js("setSpeaking(false)");
});
                    }

                    @Override
                    public void onError(String id) {
                        ui.post(() -> { speaking = false; js("setSpeaking(false)");
});
                    }
                });
                ttsReady = true;
            }
        });
    }

    void js(final String code) {
        ui.post(() -> {
            if (web != null) web.evaluateJavascript(code, null);
        });
    }
```

```
    void onTap() {
        if (speaking) {
            if (tts != null) tts.stop();
            speaking = false;
            js("setSpeaking(false)");
        } else if (!busy) {
            listen();
        }
    }

    void listen() {
        if (!SpeechRecognizer.isRecognitionAvailable(this)) {
            js("setStatus('इस फ़ोन में आवाज़ पहचान उपलब्ध नहीं है')");
            return;
        }
        if (rec == null) {
            rec = SpeechRecognizer.createSpeechRecognizer(this);
            rec.setRecognitionListener(new RecognitionListener() {
                public void onReadyForSpeech(Bundle p) { }
                public void onBeginningOfSpeech() { }
                public void onRmsChanged(float v) { }
                public void onBufferReceived(byte[] b) { }
                public void onEndOfSpeech() { }
                public void onPartialResults(Bundle b) { }
                public void onEvent(int t, Bundle b) { }

                public void onError(int code) {
                    String why;
                    switch (code) {
                        case 1: why = "नेटवर्क टाइमआउट"; break;
                        case 2: why = "नेटवर्क की दिक्कत"; break;
                        case 3: why = "माइक या ऑडियो की दिक्कत"; break;
                        case 4: why = "सर्वर की दिक्कत"; break;
                        case 5: why = "ऐप की अंदरूनी दिक्कत"; break;
                        case 6: why = "कुछ बोला नहीं गया"; break;
                        case 7: why = "समझ नहीं आया"; break;
                        case 8: why = "पहचान व्यस्त है"; if (rec != null) rec.cancel();
break;
                        case 9: why = "माइक की अनुमति नहीं है"; break;
                        default: why = "अज्ञात"; break;
                    }
                    js("setStatus(" + JSONObject.quote("त्रुटि " + code + ": " + why + ",
फिर दबाइए") + ")");
                }

                public void onResults(Bundle b) {
                    ArrayList<String> r =
b.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION);
                    if (r != null && !r.isEmpty()) ask(r.get(0));
```

```
                }
            });
        }
        Intent i = new Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH);
        i.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
        i.putExtra(RecognizerIntent.EXTRA_LANGUAGE, "hi-IN");
        js("setStatus('सुन रही हूँ…')");
        rec.startListening(i);
    }

    JSONObject msg(String role, String text) {
        try {
            return new JSONObject().put("role", role)
                    .put("parts", new JSONArray().put(new JSONObject().put("text",
text)));
        } catch (JSONException e) {
            return new JSONObject();
        }
    }

    void ask(final String text) {
        busy = true;
        js("setStatus('सोच रही हूँ…')");
        synchronized (history) {
            history.add(msg("user", text));
        }
        net.execute(() -> {
            String reply;
            try {
                reply = callGemini();
            } catch (Exception e) {
                reply = "ERR:कनेक्शन में दिक्कत है।";
            }
            final String r = reply;
            ui.post(() -> {
                busy = false;
                boolean err = r.startsWith("ERR:");
                final String shown = err ? r.substring(4) : r;
                synchronized (history) {
                    if (err) {
                        if (!history.isEmpty()) history.remove(history.size() - 1);
                    } else {
                        history.add(msg("model", r));
                    }
                }
                js("setText(" + JSONObject.quote(shown) + ")");
                if (!err && ttsReady) tts.speak(shown, TextToSpeech.QUEUE_FLUSH,
null, "miya");
            });
```

```
        });
    }

    String callGemini() throws Exception {
        JSONObject body = new JSONObject();
        body.put("systemInstruction", new JSONObject().put("parts", new
JSONArray().put(new JSONObject().put("text", PERSONA))));
        JSONArray contents = new JSONArray();
        synchronized (history) {
            int from = Math.max(0, history.size() - 20);
            for (int i = from; i < history.size(); i++) contents.put(history.get(i));
        }
        body.put("contents", contents);
        HttpURLConnection c = (HttpURLConnection) new URL(WORKER).openConnection();
        c.setRequestMethod("POST");
        c.setRequestProperty("Content-Type", "application/json");
        c.setConnectTimeout(15000);
        c.setReadTimeout(40000);
        c.setDoOutput(true);
        try (OutputStream o = c.getOutputStream()) {
            o.write(body.toString().getBytes("UTF-8"));
        }
        InputStream in = c.getResponseCode() < 400 ? c.getInputStream() :
c.getErrorStream();
        ByteArrayOutputStream bo = new ByteArrayOutputStream();
        byte[] buf = new byte[4096];
        int n;
        while ((n = in.read(buf)) > 0) bo.write(buf, 0, n);
        String resp = bo.toString("UTF-8");
        JSONObject j = new JSONObject(resp);
        JSONArray cand = j.optJSONArray("candidates");
        if (cand == null || cand.length() == 0) {
            JSONObject er = j.optJSONObject("error");
            return "ERR:जवाब नहीं मिला। " + (er != null ? er.optString("message") : "");
        }
        JSONArray parts =
cand.getJSONObject(0).getJSONObject("content").getJSONArray("parts");
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < parts.length(); i++)
sb.append(parts.getJSONObject(i).optString("text"));
        return sb.toString();
    }

    @Override
    public void onDestroy() {
        try { if (web != null) { wm.removeView(web); web.destroy(); web = null; } }
catch (Exception e) { }
        if (rec != null) rec.destroy();
        if (tts != null) tts.shutdown();
        net.shutdownNow();
```

```
        super.onDestroy();
    }
}
MIYA_EOF_6
cat > $P/app/src/main/assets/avatar.html <<'MIYA_EOF_7'
<!DOCTYPE html>
<html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<script type="importmap">
{"imports":{
"three":"https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js",
"three/addons/":"https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/",
"three/examples/jsm/":"https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/",
"@pixiv/three-vrm":"https://cdn.jsdelivr.net/npm/@pixiv/three-vrm@2.1.0/lib/three-
vrm.module.js"
}}
</script>
<style>
html,body{margin:0;height:100%;background:transparent;overflow:hidden}
#cv{width:100%;height:100%;display:block}
#bub{position:absolute;left:6px;right:6px;bottom:6px;max-
height:36%;overflow:hidden;font:13px/1.35 sans-
serif;color:#fff;background:rgba(20,24,40,.75);padding:6px 9px;border-
radius:12px;display:none}
</style></head>
<body>
<canvas id="cv"></canvas>
<div id="bub"></div>
<script type="module">
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";
import { VRMLoaderPlugin, VRMUtils } from "@pixiv/three-vrm";

const bub = document.getElementById("bub");
let hideT = null, speaking = false;
function show(t) { clearTimeout(hideT); bub.textContent = t; bub.style.display =
"block"; }
window.setStatus = t => show(t);
window.setText = t => { show(t); hideT = setTimeout(() => bub.style.display = "none",
12000); };
window.setSpeaking = b => {
  speaking = b;
  if (!b) { clearTimeout(hideT); hideT = setTimeout(() => bub.style.display = "none",
4000); }
};

const canvas = document.getElementById("cv");
const renderer = new THREE.WebGLRenderer({ canvas, alpha: true, antialias: true });
renderer.setClearColor(0x000000, 0);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
```

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(28, 1, 0.1, 20);
scene.add(new THREE.AmbientLight(0xffffff, 1.1));
const sun = new THREE.DirectionalLight(0xffffff, 1.6);
sun.position.set(1, 2, 2);
scene.add(sun);
function resize() {
  const w = canvas.clientWidth, h = canvas.clientHeight;
  renderer.setSize(w, h, false);
  camera.aspect = w / h;
  camera.updateProjectionMatrix();
}
window.addEventListener("resize", resize);
resize();

let vrm = null, head = null;
const loader = new GLTFLoader();
loader.register(p => new VRMLoaderPlugin(p));
loader.load("model.vrm", gltf => {
  vrm = gltf.userData.vrm;
  try { VRMUtils.removeUnnecessaryVertices(gltf.scene); } catch (e) {}
  VRMUtils.rotateVRM0(vrm);
  scene.add(vrm.scene);
  const h = vrm.humanoid;
  head = h.getNormalizedBoneNode("head");
  const la = h.getNormalizedBoneNode("leftUpperArm"), ra =
h.getNormalizedBoneNode("rightUpperArm");
  if (la) la.rotation.z = 1.2;
  if (ra) ra.rotation.z = -1.2;
  vrm.scene.updateMatrixWorld(true);
  const p = new THREE.Vector3();
  head.getWorldPosition(p);
  camera.position.set(0, p.y - 0.05, 1.3);
  camera.lookAt(0, p.y - 0.1, 0);
}, undefined, () => show("model.vrm नहीं मिला"));

let nextBlink = 2, blinkT = -1;
const clock = new THREE.Clock();
function loop() {
  requestAnimationFrame(loop);
  const dt = clock.getDelta(), t = clock.elapsedTime;
  if (vrm) {
    const em = vrm.expressionManager;
    const target = speaking ? 0.25 + 0.45 * Math.abs(Math.sin(t * 13)) + 0.15 *
Math.random() : 0;
    const cur = em.getValue("aa") || 0;
    em.setValue("aa", cur + (target - cur) * 0.35);
    if (blinkT < 0 && t > nextBlink) blinkT = 0;
    if (blinkT >= 0) {
      blinkT += dt;
```

```
      em.setValue("blink", Math.max(0, Math.sin(Math.min(blinkT / 0.18, 1) *
Math.PI)));
      if (blinkT > 0.18) { blinkT = -1; nextBlink = t + 2 + Math.random() * 3; }
    }
    if (head) { head.rotation.y = Math.sin(t * 0.6) * 0.06; head.rotation.x =
Math.sin(t * 0.9) * 0.025; }
    vrm.update(dt);
  }
  renderer.render(scene, camera);
}
loop();
</script>
</body></html>
MIYA_EOF_7
if [ -f model.vrm ]; then cp model.vrm $P/app/src/main/assets/model.vrm; else echo
"model.vrm repo me nahi mila"; exit 1; fi
```